POLITECHNIKA MORSKA – WYDZIAŁ INFORMATYKI

PARADYGMATY PROGRAMOWANIA – LAB 05 Programowanie strukturalne – funkcje, rekurencja

Napisz funkcję `silnia_iter(unsigned int n)` i `silnia_rek(unsigned int n)`, która zwróci wartość silni liczby `n`. (rekurencyjnie i iteracyjnie)

Napisz funkcję `suma_iter(unsigned int n)` i `suma_rek(unsigned int n)`, która zwróci sumę cyfr liczby `n`. (rekurencyjnie i iteracyjnie)

Napisz funkcję `fibon_iter(unsigned int n)` która zwróci `n-tą` wartość ciągu Fibonacciego, obliczoną metodą iteracyjną.

Napisz funkcję `fibon_rek(unsigned int n)` która zwróci `n-tą` wartość ciągu Fibonacciego, obliczoną metodą rekurencyjną.

„Porównanie czasu działania dwóch implementacji funkcji obliczającej silnię: iteracyjnej i rekurencyjnej, oraz ocena wpływu sposobu implementacji na efektywność czasową i pamięciową programu."

Korzystając z biblioteki **chrono** i odpowiednio dobranych wartości **n** przeanalizuj i porównaj szybkość działania funkcji iteracyjnych i rekurencyjnych. Przygotuj sprawozdanie, w którym zawarte będą informacje:

- Czy podejście rekurencyjne i iteracyjne jest w stanie obliczyć takie same maksymalne wartości tych funkcji? Jakie maksymalne wartości można obliczyć bez przekraczania zakresu wybranego typu danych?
- Czy przy obliczeniach czas działania funkcji rekurencyjnych i iteracyjnych jest taki sam? Dla jakich zakresów **n** lepsza jest jedna implementacja, a dla jakich druga?
- Jak zmienia się czas działania programów rekurencyjnych i iteracyjnych przy obliczaniu małych wartości i dużych wartości? Przemyśl i przygotuj odpowiednie zakresy danych (np. małe wartości **n** od 1 do np. 10, średnie wartości **n** od 15 do np. 20, duże wartości **n**)
- Przygotuj wykres czasu w zależności od parametrów dla rekurencji i iteracji.

Wskazówki:

- W sprawozdaniu dobrze jest uzasadnić wybór zakresu. Dla silni nie bada się zwykle bardzo dużych **n** ze względu na: przepełnienie typu i rosnące ryzyko przepełnienia stosu przy rekurencji.
- Dla każdej wartości **n** należy uruchomić funkcję iteracyjną wiele razy, uruchomić funkcję rekurencyjną tyle samo razy, zapisać łączny czas lub średni czas jednego wywołania. To eliminuje wpływ przypadkowych zakłóceń systemowych.
- Podczas pomiarów należy uruchamiać oba testy w tym samym programie, używać tych samych danych wejściowych, nie wykonywać zbędnych operacji w sekcji mierzonej, nie wypisywać wyników na ekran w czasie pomiaru. Operacje wejścia/wyjścia bardzo silnie zaburzają wyniki.
- Podczas pomiarów należy uruchamiać oba testy w tym samym programie, używać tych samych danych wejściowych, nie wykonywać zbędnych operacji w sekcji mierzonej, nie wypisywać wyników na ekran w czasie pomiaru. Operacje wejścia/wyjścia bardzo silnie zaburzają wyniki.
- Dla każdej wartości **n** warto zapisać **n**, czas funkcji iteracyjnej, czas funkcji rekurencyjnej, różnicę czasu, ewentualnie stosunek czasów.
- Należy opisać: środowisko testowe, sposób pomiaru czasu, liczbę powtórzeń, zakres testowanych wartości **n**, sposób obliczania wyniku końcowego.
- Analiza wyników - tutaj należy odpowiedzieć która funkcja była szybsza, czy i jak różnica rosła wraz ze wzrostem **n**, z czego wynika przewaga jednej z metod. W jakich zakresach wartości **n** jedna z funkcji była efektywniejsza.
- Dobór zakresu wartości n jest jednym z ważniejszych elementów całego eksperymentu, ponieważ bezpośrednio wpływa na wiarygodność porównania oraz na możliwość poprawnej interpretacji wyników, dlatego zakres testów nie może być dobrany przypadkowo. Jeżeli wybierze się wyłącznie bardzo małe wartości n, różnice czasowe między funkcją iteracyjną i rekurencyjną mogą być tak małe, że praktycznie niewidoczne. Z kolei zbyt duże wartości n mogą prowadzić do przekroczenia zakresu typu liczbowego, błędnych wyników, a w przypadku rekurencji nawet do przepełnienia stosu. Dlatego zakres danych wejściowych powinien być dobrany tak, aby umożliwić wykonanie poprawnych obliczeń, pozwolić uchwycić różnice czasowe, nie zaburzyć wyników błędami technicznymi.
- Najlepiej nie wybierać wartości n losowo, lecz podzielić je na grupy. Dzięki temu analiza wyników jest bardziej uporządkowana. Małe wartości n - przykładowo: n = 0, 1, 2, 3, 5 Taki zakres pozwala sprawdzić poprawność działania funkcji, zweryfikować przypadki bazowe, upewnić się, że obie implementacje zwracają identyczne wyniki. Małe wartości są szczególnie przydatne na początku eksperymentu, ale zwykle nie pokazują jeszcze wyraźnych różnic czasowych. Tak samo dla średnich i dużych wartości.
- Zakres n proszę dobrać używając wartości kolejnych: n = 1, 2, 3, ... Taki dobór jest korzystny, gdy zakres jest niewielki, zależy nam na dokładnym wykresie, chcemy zaobserwować zmianę czasu niemal punkt po punkcie.
- Zbyt duże wartości n są problematyczne, ponieważ wynik funkcji bardzo szybko przekraczą zakres typu, rekurencja zwiększa zużycie stosu, pomiar może dotyczyć już działania programu w warunkach błędnych, analiza przestaje dotyczyć samego algorytmu, a zaczyna dotyczyć ograniczeń technicznych środowiska. Dlatego lepiej dobrać **umiarkowany, kontrolowany zakres**, zamiast próbować badać skrajnie duże wartości.